

Laporan Tugas Kecil 1 IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Penyelesaian Permainan *Queens* LinkedIn

Arina Azka and 13524049¹

¹13524049@std.stei.itb.ac.id, ²13524049@gmail.com

Queens LinkedIn adalah permainan logika yang tersedia pada situs LinkedIn. Tujuan utama permainan ini adalah menempatkan tepat satu *Queen* pada tiap baris, kolom, dan daerah warna di dalam sebuah papan persegi berwarna.

Permainan *Queen LinkedIn* dapat dikatakan sebagai pengembangan dari persoalan klasik *N Queen Problem*. Meskipun memiliki dasar yang serupa, *Queens* LinkedIn menerapkan beberapa aturan tambahan berupa pembagian wilayah warna dan aturan ketentanggaan yang spesifik. Berikut ini adalah aturan permainan *Queens* LinkedIn.

- a. Hanya terdapat tepat satu *Queen* pada setiap baris.
- b. Hanya terdapat tepat satu *Queen* pada setiap kolom.
- c. Hanya terdapat tepat satu *Queen* pada setiap wilayah warna.
- d. Beberapa *Queen* dapat terletak pada garis diagonal yang sama selama posisinya tidak tepat bersebelahan.

Laporan kali ini menyajikan pendekatan komputasi untuk menyelesaikan permainan *Queens* LinkedIn menggunakan algoritma brute force murni yang dituliskan dalam bahasa pemrograman Python.

Algoritma Brute Force

Algoritma brute force merupakan algoritma sederhana yang menyelesaikan permasalahan menggunakan strategi lampang atau naif. Terdapat dua pendekatan Algoritma brute force yang diterapkan pada laporan ini, yaitu *exhaustive search* dan *backtracking*.

a. *Exhaustive search*

Algoritma brute force pertama yang diterapkan dalam laporan ini diklasifikasikan sebagai algoritma brute force murni atau *exhaustive search*. *Exhaustive search* didefinisikan sebagai metode pencarian sistematis yang mengeksplorasi setiap opsi hingga jawaban suatu masalah ditemukan. Pendekatan ini menjadi algoritma *default* yang diterapkan pada aplikasi ketika pengguna menonaktifkan toggle “Enable Optimization”.

Exhaustive search umumnya diterapkan dengan membuat semua opsi, lalu memeriksa apakah opsi tersebut adalah solusi yang valid atau bukan. Dengan mempertimbangkan jumlah opsi yang sangat besar dan tumbuh secara eksponensial, penulis melakukan sedikit modifikasi. Pada penerapannya, aplikasi memeriksa setiap opsi, yaitu sebuah papan utuh dengan seluruh *Queen* diletakkan, tepat setelah opsi tersebut dibuat menggunakan proses rekursi. Meskipun pengecekan per kolom pada sebuah papan dihentikan tepat setelah papan tersebut dikonfirmasi salah, pendekatan ini tetap dikategorikan sebagai *exhaustive search*. Hal ini dikarenakan tidak adanya mekanisme *pruning* (pemangkasan) di tengah proses pembentukan opsi, sehingga algoritma tetap dipaksa mengunjungi setiap cabang hingga kedalaman maksimal.

b. *Backtracking*

Berbeda dengan *exhaustive search*, pendekatan *backtracking* pada aplikasi ini mengintegrasikan mekanisme *pruning* atau pemangkasan ruang pencarian. Dengan mengacu pada algoritma sudoku yang terdapat pada halaman 45 PPT Brute-Force Bagian.1 2025/2026 ([https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-(2026)-Bag1.pdf)), optimasi dilakukan dengan cara memvalidasi setiap langkah penempatan *Queen* secara *real-time* di setiap level rekursi

Backtracking adalah algoritma pemecahan masalah yang melibatkan pencarian solusi secara bertahap dengan mencoba berbagai pilihan dan membatalkannya jika pilihan tersebut mengarah ke jalan buntu. Pendekatan ini menjadi opsi algoritma kedua dan dapat digunakan pengguna ketika mengaktifkan toggle “Enable Optimization”.

Jika pada suatu langkah penempatan *Queen* ditemukan pelanggaran aturan, seperti pelanggaran aturan baris, wilayah warna, atau posisi tetangga, maka algoritma akan segera menghentikan penelusuran pada cabang tersebut tanpa harus menunggu papan terisi penuh. Proses ‘mundur’ atau *backtrack* ini dilakukan untuk mencoba kemungkinan angka atau posisi lain pada kolom sebelumnya. Dengan membuang ribuan hingga jutaan jalur yang sudah pasti tidak mengarah pada solusi, *backtracking* secara drastis mengurangi total kasus yang perlu diperiksa. Oleh karena itu, pendekatan ini menawarkan waktu eksekusi yang jauh lebih efisien bagi pengguna, terutama pada ukuran papan yang kompleks.

Analisis dan Rancangan

a. Batasan Masalah

Untuk memastikan aplikasi berjalan sesuai dengan aturan permainan *Queens LinkedIn*, ditetapkan batasan-batasan berikut:

- **Struktur Papan**

Papan harus berbentuk persegi dengan n jumlah baris sama dan n jumlah kolom.

- **Daerah Warna**

Setiap papan terdiri dari n wilayah warna yang berbeda, ditandai dengan satu huruf kapital (A-Z).

b. Representasi Persoalan

Persoalan dimodelkan menggunakan beberapa struktur data utama dalam bahasa pemrograman Python, antara lain:

- **Matriks Papan (matrix)**

List dua dimensi yang menyimpan karakter huruf untuk merepresentasikan pembagian wilayah warna pada papan.

- **Array Solusi (current_p)**

List satu dimensi berukuran n dengan indeks list merepresentasikan nomor kolom, dan nilai di dalamnya merepresentasikan nomor baris tempat *Queen* diletakkan.

- **Pemetaan Warna (color_map)**

Dictionary yang memetakan setiap karakter wilayah ke nilai warna RGB untuk visualisasi gambar.

c. Inisiasi

Proses inisiasi merupakan langkah awal untuk menyiapkan data sebelum algoritma pencarian dijalankan.

1) Input Data

Program menerima masukan berupa berkas .txt yang berisi matriks karakter kapital (A-Z) yang merepresentasikan wilayah warna pada papan.

2) Validasi Struktur

Memeriksa apakah matriks tersebut berbentuk persegi. Jika jumlah baris tidak sama dengan jumlah kolom, jumlah warna tidak sama dengan ukuran papan, atau terdapat karakter yang tidak valid, program akan menampilkan pesan kesalahan.

3) Penentuan Skala

Menghitung nilai n (ukuran papan) yang juga menentukan jumlah *Queen* yang harus diletakkan serta jumlah wilayah warna yang harus unik.

4) Konfigurasi Visual

Menyiapkan *color_map* untuk memetakan setiap huruf wilayah ke warna RGB tertentu guna keperluan *rendering* pada antarmuka grafis.

5) **Penyiapan Wadah Solusi**

Menginisialisasi list kosong *current_p* (yang akan menampung posisi *Queen* selama proses pencarian berlangsung).

d. **Prosedur *Exhaustive search***

Pendekatan ini berfokus pada pembangkitan opsi susunan papan sebelum melakukan validasi akhir. Prosedur ini diterapkan melalui fungsi *recursiveWithoutPruning* dengan mekanisme sebagai berikut.

- 1) Mulai dari kolom pertama dan coba letakkan *Queen* pada setiap baris (0 hingga $n-1$) tanpa melakukan pengecekan aturan di tengah proses.
- 2) Lanjutkan proses penempatan *Queen* ke kolom berikutnya secara rekursif hingga seluruh kolom (n) terisi oleh *Queen*.
- 3) Setiap kali satu susunan papan utuh terbentuk, lakukan pengecekan validitas secara menyeluruh menggunakan fungsi *isSolution*.
- 4) Lakukan pengecekan untuk memastikan susunan memenuhi seluruh aturan yang telah ditetapkan.
- 5) Jika susunan tersebut valid, maka solusi ditemukan dan proses dihentikan.
- 6) Jika susunan tidak valid, abaikan dan lanjutkan untuk membangkitkan kombinasi angka baris berikutnya pada kolom terakhir.

e. **Prosedur *Backtracking***

Pendekatan ini mengoptimalkan pencarian dengan melakukan validasi di setiap langkah penempatan *Queen*.

1. Mulai dari kolom pertama dan coba letakkan *Queen* pada baris tertentu di kolom tersebut.
2. Untuk setiap penempatan, lakukan validasi *real-time* untuk memastikan posisi tersebut memenuhi aturan.
3. Jika posisi ditemukan aman, letakkan *Queen* dan segera berlanjut ke kolom berikutnya untuk mencari posisi *Queen* selanjutnya.
4. Jika posisi tidak aman, coba letakkan *Queen* pada baris berikutnya di kolom yang sama.
5. Jika tidak ditemukan baris yang aman hingga baris terakhir di kolom tersebut, lakukan *backtracking*, yaitu hapus *Queen* di kolom saat ini dan kembali ke kolom sebelumnya untuk mencoba baris selanjutnya di kolom tersebut.

f. Rancangan GUI

Antarmuka pengguna dibangun menggunakan pustaka `customtkinter` dengan pembagian *section* sebagai berikut.

- **Control Section**

Terdiri dari tombol *Import File* untuk masukan data `.txt`, *toggle* "Enable Optimization" untuk memilih algoritma, serta tombol *Solve*, *Save*, dan *Reset*.

- **Output Section**

- **Frame Visualisasi**

Merender matriks papan menjadi gambar berwarna dan menampilkan posisi *Queen* secara *real-time*.

- **Frame Statistik**

Menampilkan kinerja aplikasi berupa *Execution Time* dan *Total Cases Checked* yang diperbarui secara *real-time*.

Implementasi

Berikut ini penjelasan fungsi-fungsi yang digunakan dengan mengelompokkannya ke dalam beberapa modul.

a. *Queens_linked.py*

Modul ini berisi algoritma inti dari aplikasi karena menangani logika utama untuk menyelesaikan permainan *Queens* LinkedIn menggunakan dua pendekatan, yaitu *Backtracking* dan *Exhaustive search*. Modul ini bertanggung jawab untuk memvalidasi papan, memeriksa aturan penempatan *Queen*, serta menyediakan mekanisme *callback* untuk visualisasi proses pencarian secara *real-time*.

1) **isValid(matrix)**

Fungsi ini adalah validasi awal untuk memastikan papan memenuhi syarat teknis permainan, yaitu papan tidak kosong, papan berbentuk persegi, setiap elemen adalah huruf kapital (representasi warna), dan jumlah warna unik harus tepat sama dengan ukuran papan.

2) **isSafe(current_p, new_row, colored_board)**

Fungsi menentukan apakah *Queen* baru bisa diletakkan di baris tertentu pada kolom saat ini tanpa melanggar aturan. Beberapa aturan yang diperiksa adalah tidak ada *Queen* di baris yang sama, tidak ada *Queen* di wilayah warna yang sama, dan *Queen* tidak bersinggungan langsung secara vertikal atau diagonal dengan *Queen* di kolom sebelumnya.

3) **isSolution(p, colored_board)**

Fungsi yang digunakan oleh metode *exhaustive search* untuk mengecek apakah susunan *Queen* yang sudah lengkap memenuhi semua kriteria keamanan.

4) **recursiveWithPruning(n, current_p, colored_board)**

Fungsi implementasi algoritma *backtracking* yang mencoba meletakkan *Queen* kolom demi kolom. Jika di tengah jalan ditemukan posisi yang tidak aman (*isSafe* bernilai *False*), algoritma akan langsung berhenti di cabang tersebut (*pruning*) dan mencoba kemungkinan lain. Fungsi ini juga memberikan *visual_callback* untuk memperbarui tampilan GUI.

5) **recursiveWithoutPruning(n, current_p, colored_board)**

Fungsi implementasi algoritma *exhaustive search* yang mencoba semua kemungkinan posisi *Queen* tanpa memedulikan aturan di tengah proses. Aturan hanya dicek menggunakan *isSolution* setelah semua *Queen* diletakkan.

6) **findFirstSolution(n, colored_board, use_pruning=True)**

Fungsi untuk menginisialisasi penghitung kasus (*i* = 0) dan memilih metode pencarian berdasarkan parameter *use_pruning* yang dipilih pengguna.

```
visual_callback = None

# Fungsi untuk memeriksa apakah papan yang diberikan valid
def isValid(matrix):
    # Papan kosong tidak valid
    if not matrix:
        return False

    num_rows = len(matrix)
    unique_colors = set()
    for row in matrix:
        # Memeriksa apakah papan persegi
        if len(row) != num_rows:
            return False

        # Memeriksa apakah setiap elemen adalah huruf kapital
        for char in row:
            if not isinstance(char, str) or not char.isalpha() or not char.isupper():
                return False
            unique_colors.add(char)

    if len(unique_colors) != num_rows:
        return False
```

```

    return True

# Fungsi untuk memeriksa apakah aman untuk meletakkan queen pada baris
new_row
def isSafe(current_p, new_row, colored_board):
    current_col = len(current_p)

    new_color = colored_board[new_row][current_col]

    # Melakukan iterasi dari kolom pertama hingga kolom terakhir
    # sebelum kolom yang akan diisi
    for prev_col, queen_row in enumerate(current_p):
        # Cek Baris
        if queen_row == new_row:
            return False

        # Cek Warna
        if colored_board[queen_row][prev_col] == new_color:
            return False

    # Cek apakah queen pada kolom sebelumnya bersinggungan dengan
    # queen sekarang
    if current_col > 0:
        prev_row = current_p[current_col - 1]
        if abs(prev_row - new_row) == 1:
            return False

    return True

# Fungsi untuk memeriksa apakah susunan queen pada papan aman
def isSolution(p, colored_board):
    region_used = set()
    current_p = []
    for col in range(len(p)):
        row = p[col]

        if not isSafe(current_p, row, colored_board):
            return False

        current_p.append(row)
    return True

# Fungsi yang menyelesaikan permainan Queens LinkedIn secara rekursif
# dengan pruning (backtracking)
def recursiveWithPruning(n, current_p, colored_board):
    global i, visual_callback

    if len(current_p) == n:

```

```

        return current_p

    for row in range(n):
        i+=1

        if i % 100000 == 0:
            visual_callback(current_p + [row])
        if isSafe(current_p, row, colored_board):
            result = recursiveWithPruning(n, current_p + [row],
colored_board)

            if result is not None:
                return result

    return None

# Fungsi yang menyelesaikan permainan Queens LinkedIn secara rekursif
tanpa pruning (exhaustive search)
def recursiveWithoutPruning(n, current_p, colored_board):
    global i, visual_callback
    if len(current_p) == n:

        if isSolution(current_p, colored_board):
            return current_p
        return None

    for row in range(n):
        i+=1

        if i % 25000000 == 0:
            visual_callback(current_p + [row])
        result = recursiveWithoutPruning(n, current_p + [row],
colored_board)
        if result is not None:
            return result

    return None

# Fungsi untuk menemukan susunan queen pertama yang aman
def findFirstSolution(n, colored_board, use_pruning=True):
    global i
    i = 0
    if use_pruning:
        return recursiveWithPruning(n, [], colored_board)
    else:
        return recursiveWithoutPruning(n, [], colored_board)

```

b. file_processing.py

Modul `file_processing.py` bertanggung jawab atas seluruh operasi Input/Output (I/O) yang berkaitan dengan manajemen berkas teks dalam aplikasi.

1) `readFile(filename)`

Untuk mengonversi data dari berkas teks mentah menjadi matriks yang dapat diolah oleh algoritma.

2) `matrixToImage(matrix, solution, save_path, color_map)`

Bertugas membuat gambar papan permainan berdasarkan skema warna dan posisi *Queen*. Fungsi ini menggunakan `color_map` untuk memberikan warna unik pada setiap daerah warna di papan yang diwakili oleh huruf A-Z.

```
import numpy as np
from PIL import Image, ImageDraw
import os

# Fungsi untuk membaca file sebagai matrix
def readFile(filename):
    matrix = []
    with open(filename) as f:
        for line in f:
            row = list(line.strip())
            matrix.append(row)

    return matrix

color_map = {
    'A': [109, 158, 235],
    'B': [147, 196, 125],
    'C': [255, 153, 0],
    'D': [142, 124, 195],
    'E': [224, 102, 102],
    'F': [255, 217, 102],
    'G': [230, 184, 175],
    'H': [111, 168, 220],
    'I': [255, 123, 172],
    'J': [118, 165, 175],
    'K': [180, 167, 214],
    'L': [213, 166, 189],
    'M': [160, 190, 50],
    'N': [61, 133, 198],
    'O': [166, 77, 121],
    'P': [246, 178, 107],
    'Q': [204, 0, 0],
    'R': [69, 129, 142],
    'S': [106, 168, 79],
    'T': [159, 197, 232],
```

```

        'U': [0, 160, 160],
        'V': [249, 203, 156],
        'W': [182, 215, 168],
        'X': [207, 226, 243],
        'Y': [217, 217, 217],
        'Z': [234, 209, 220]
    }

    QUEEN_ICON = None

    # Fungsi untuk mengubah matriks menjadi gambar
    def matrixToImage(matrix, solution, save_path, color_map):
        global QUEEN_ICON
        n = len(matrix)
        cell_size = 100
        queen_size = int(1/2*cell_size)

        warna = np.zeros((n, n, 3), dtype=np.uint8)
        img = Image.new("RGB", (n*cell_size, n*cell_size), "white")
        draw = ImageDraw.Draw(img)

        if QUEEN_ICON is None:
            if os.path.exists("src/assets/queen_icon.png"):
                raw_img = Image.open("src/assets/queen_icon.png").convert("RGBA")
                QUEEN_ICON = raw_img.resize((queen_size, queen_size), Image.LANCZOS)
            else:
                print("File queen_icon.png not found!")

        for row in range(n):
            for col in range(n):
                x, y = col * cell_size, row * cell_size

                huruf = matrix[row][col]
                warna = tuple(color_map.get(huruf, [255, 255, 255]))
                draw.rectangle([x, y, x + cell_size, y + cell_size],
                    fill=warna, outline="black", width=2)

                if solution and len(solution) > col and solution[col] == row:
                    if QUEEN_ICON:
                        offset = (cell_size - queen_size) // 2
                        img.paste(QUEEN_ICON, (x + offset, y + offset), QUEEN_ICON)
                    else:
                        draw.ellipse([x+20, y+20, x+80, y+80],
                            fill="black")

```

```

if save_path:
    if not save_path.lower().endswith(".png"):
        save_path += ".png"
    img.save(save_path)

return img

```

c. **main.py**

Modul `main_gui.py` merupakan antarmuka grafis utama dari aplikasi *Queens* LinkedIn. Modul ini dibangun menggunakan pustaka `customtkinter`. Peran utamanya adalah sebagai pengendali yang menghubungkan interaksi pengguna dengan logika pemrosesan data dan algoritma pencarian. Modul ini juga menerapkan konsep `multithreading` agar antarmuka tetap responsif saat algoritma sedang bekerja.

1) **importFile()**

Bertugas menangani proses pengunggahan berkas konfigurasi papan (.txt). Apabila berkas berhasil dibuka, fungsi ini memperbarui status tombol "Solve", "Save", dan "Reset" menjadi aktif.

2) **solveHandler() dan solveGame()**

Keduanya berperan dalam memulai proses pencarian solusi. `SolveHandler()` membuat thread baru agar proses pencarian tidak membekukan aplikasi, sedangkan `solveGame()` memicu fungsi `findFirstSolution()`, mengatur mekanisme monitoring iterasi, dan menangani perubahan status label status permainan.

3) **updateBoardDisplay(solution, is_preview)**

Bertugas menampilkan gambar papan secara dinamis ke area visualisasi. Gambar papan yang ditampilkan berupa pratinjau papan yang baru diunggah maupun menampilkan progres papan saat algoritma sedang berjalan hingga selesai.

4) **resetHandler(opt_switch)**

Apabila ditekan, fungsi ini mengembalikan aplikasi ke kondisi awal, yaitu tidak ada papan yang sudah diunggah, mematikan tombol-tombol fungsional, dan mengembalikan statistik kinerja aplikasi menjadi 0.

5) **saveResultHandler()**

Fungsi ini mampu mengunduh gambar papan yang ditampilkan pada area visualisasi ke dalam format .png, baik papan awal ataupun hasil solusi.

6) **Inisiasi GUI**

Bertugas menyiapkan kerangka utama aplikasi, mengatur tema, dan resolusi jendela

7) **Fungsi Pembangun Antarmuka**

Fungsi-fungsi ini digunakan untuk menyusun tata letak (layout) aplikasi secara modular.

a) header(main_frame)

Menampilkan judul utama dan identitas pembuat (Arina Azka - 13524049).

b) controlSection(main_frame)

Membangun panel sisi kanan yang berisi tombol Import, toggle optimasi (Enable Optimization), tombol Solve, Save, dan Reset.

c) outputSection(main_frame)

Wadah untuk bagian statistik dan visualisasi di sisi kiri.

- **visualSection(output_frame)**

Menyiapkan area bingkai tempat gambar papan akan ditampilkan.

- **statsSection(output_frame)**

Menyiapkan label untuk menampilkan "Execution Time" dan "Total Cases Checked".

d) mainApp()

Berfungsi untuk dan menggunakan header(), contentSection(), dan main_frame untuk menyusun tata letak. Fungsi pengikat akhir yang menyusun seluruh bagian (header dan content) ke dalam kontainer pusat aplikasi.

```
import time
import threading
import os
import customtkinter as ctk
from tkinter import filedialog, messagebox
from PIL import Image

from modules.file_processing import *
from modules.queens_linkedin import *

CURRENT_BOARD = None
LAST_SOLUTION = []

IS_RUNNING = False
SEARCHING_STATUS = None
SEARCHING_TIME = None
CASES_CHECKED = None

DISPLAY_AREA = None
SOLVE_BUTTON = None

SAVE_BUTTON = None
```

```

RESET_BUTTON = None
OPT_SWITCH = None

def updateBoardDisplay(solution=[], is_preview=True):
    global CURRENT_BOARD, DISPLAY_AREA, LAST_SOLUTION

    if CURRENT_BOARD is not None:
        pil_img = matrixToImage(CURRENT_BOARD, solution, None,
color_map)
        LAST_SOLUTION = solution

        for child in DISPLAY_AREA.winfo_children():
            child.destroy()

        ctk_img = ctk.CTkImage(light_image=pil_img,
dark_image=pil_img, size=(280, 280))

        board_label = ctk.CTkLabel(DISPLAY_AREA, image=ctk_img,
text="")
        board_label.place(relx=0.5, rely=0.5, anchor="center")

def importFile():
    global CURRENT_BOARD, SOLVE_BUTTON, SAVE_BUTTON, LAST_SOLUTION,
SEARCHING_STATUS, RESET_BUTTON
    file_path = filedialog.askopenfilename(title="Select a file",
filetypes=[("Text files", "*.txt")])

    if file_path:
        board = readFile(file_path)
        if isValid(board):
            CURRENT_BOARD = board
            updateBoardDisplay()

            SEARCHING_STATUS.configure(text="Press Solve button to
find solution", text_color="white")
            SEARCHING_TIME.configure(text="0")
            CASES_CHECKED.configure(text="0")

            SOLVE_BUTTON.configure(state="normal", fg_color="#7b68ee")
            SAVE_BUTTON.configure(state="normal",
text_color="#7b68ee", border_color="#7b68ee")
            RESET_BUTTON.configure(state="normal", text_color="white",
border_color="grey")
        else:
            messagebox.showerror("Error", "Board is invalid!")

```

```

def solveHandler():
    threading.Thread(target=solveGame, daemon=True).start()

def solveGame():
    global CURRENT_BOARD, SEARCHING_TIME, CASES_CHECKED,
    SEARCHING_STATUS, SAVE_BUTTON

    if CURRENT_BOARD is None:
        return

    SEARCHING_STATUS.configure(text="Searching for solution...",
text_color="#7b68ee")

    n = len(CURRENT_BOARD)
    start_time = time.time()

    import modules.queens_linked_in
    modules.queens_linked_in.i = 0

    modules.queens_linked_in.visual_callback = lambda progress:
app.after(0, updateBoardDisplay, progress, False)

    stop_monitoring = False

    def monitor():
        while not stop_monitoring:
            elapsed = time.time() - start_time
            current_nodes = modules.queens_linked_in.i

            SEARCHING_TIME.configure(text=f"{elapsed:.4f} s")
            CASES_CHECKED.configure(text=f"{current_nodes:,}")

    threading.Thread(target=monitor, daemon=True).start()

    is_optimized = OPT_SWITCH.get()
    solution = findFirstSolution(n, CURRENT_BOARD, is_optimized)

    stop_monitoring = True

    final_nodes = modules.queens_linked_in.i
    final_time = time.time() - start_time

    SEARCHING_TIME.configure(text=f"{final_time:.4f} s")
    CASES_CHECKED.configure(text=f"{final_nodes:,}")

    modules.queens_linked_in.visual_callback = None

    SAVE_BUTTON.configure(

```

```

        state="normal",
        text_color="#7b68ee",
        border_color="#7b68ee"
    )
    if solution:
        SEARCHING_STATUS.configure(text="Solution found!",
text_color="#2ecc71")
        app.after(0, updateBoardDisplay, solution, False)
    else:
        SEARCHING_STATUS.configure(text="Solution not found.",
text_color="#e74c3c")
        app.after(0, updateBoardDisplay, [], True)

def saveResultHandler():
    global CURRENT_BOARD, LAST_SOLUTION
    if CURRENT_BOARD is None: return

    file_path = filedialog.asksaveasfilename(
        defaultextension=".png",
        filetypes=[("PNG files", "*.png")],
        title="Save Solution As"
    )

    if file_path:
        matrixToImage(CURRENT_BOARD, LAST_SOLUTION, file_path,
color_map)
        messagebox.showinfo("Success", "Image saved successfully!")

def resetHandler(OPT_SWITCH):
    global CURRENT_BOARD, IS_RUNNING, stop_monitoring, SAVE_BUTTON,
RESET_BUTTON

    IS_RUNNING = False
    stop_monitoring = True

    import modules.queens_linked_in
    modules.queens_linked_in.visual_callback = None

    CURRENT_BOARD = None
    SOLVE_BUTTON.configure(state="disabled", fg_color="gray")
    SAVE_BUTTON.configure(state="disabled", text_color="gray",
border_color="gray")
    RESET_BUTTON.configure(state="disabled", text_color="gray",
border_color="gray")
    if OPT_SWITCH:
        OPT_SWITCH.deselect()

    SEARCHING_TIME.configure(text="0")

```

```

CASES_CHECKED.configure(text="0")
SOLVE_BUTTON.configure(state="disabled", fg_color="gray")

for child in DISPLAY_AREA.winfo_children():
    child.destroy()
placeholder = ctk.CTkLabel(DISPLAY_AREA, text="No board loaded
yet", text_color="gray", font=("Poppins", 12))
placeholder.place(relx=0.5, rely=0.5, anchor="center")

for f in ["input_board.png", "realtime_progress.png"]:
    if os.path.exists(f):
        try: os.remove(f)
        except: pass

if SEARCHING_STATUS:
    SEARCHING_STATUS.configure(text="Upload a file to start
searching", text_color="gray")

def header(main_frame):
    title_label = ctk.CTkLabel(main_frame, text="Queens LinkedIn",
font=ctk.CTkFont("Poppins", size=40, weight="bold"))
    title_label.pack(padx=10, pady=(40, 5))
    subtitle_label = ctk.CTkLabel(main_frame, text="Arina Azka -
13524049", font=ctk.CTkFont("Poppins", size=16))
    subtitle_label.pack(padx=0, pady=(0, 20))

def contentSection(main_frame):
    content_frame = ctk.CTkFrame(main_frame, fg_color="transparent")
    content_frame.pack(pady=10, padx=20)

    controlSection(content_frame)
    outputSection(content_frame)

def controlSection(content_frame):
    global SOLVE_BUTTON, OPT_SWITCH, SAVE_BUTTON, RESET_BUTTON
    control_frame = ctk.CTkFrame(content_frame, corner_radius=8,
width=280, height=480)
    control_frame.pack(side="right", padx=10, pady=10)
    control_frame.pack_propagate(False)

    upload_label = ctk.CTkLabel(control_frame, text="Upload Board",
font=ctk.CTkFont("Poppins", 14, weight="bold"))
    upload_label.pack(pady=(20, 5), padx=20, anchor="w")

    upload_button = ctk.CTkButton(control_frame, text="Import File",
fg_color="#7b68ee",
hover_color="#6a5acd",

```



```

                                font=("Poppins", 14, "bold"),
height=32, command=importFile)
    upload_button.pack(padx=20, pady=5, fill="x")

    upload_note = ctk.CTkLabel(control_frame, text="* Only accepts
.txt files", text_color="gray", font=ctk.CTkFont("Poppins", size=12))
    upload_note.pack(padx=20, pady=(0, 10), anchor="w")

    opt_frame = ctk.CTkFrame(control_frame, fg_color="transparent")
    opt_frame.pack(fill="x", padx=20, pady=(20, 10))
    opt_label = ctk.CTkLabel(
        opt_frame,
        text="Optimization Settings",
        font=("Poppins", 14, "bold")
    )
    opt_label.pack(anchor="w")
    OPT_SWITCH = ctk.CTkSwitch(
        opt_frame,
        text="Enable Optimization",
        font=("Poppins", 12),
        progress_color="#7b68ee"
    )
    OPT_SWITCH.pack(pady=(10, 0), anchor="w")
    opt_desc = ctk.CTkLabel(
        opt_frame,
        text="Brute Force with Pruning",
        font=("Poppins", 11),
        text_color="gray"
    )
    opt_desc.pack(anchor="w")

    buttons_frame = ctk.CTkFrame(control_frame,
fg_color="transparent")

    SOLVE_BUTTON = ctk.CTkButton(buttons_frame, text="▷ Solve",
state="disabled",
                                fg_color="gray", hover_color="#6a5acd",
                                height=32, font=("Poppins", 14, "bold"),
command=solveHandler)
    SOLVE_BUTTON.pack(padx=20, pady=(20, 5), fill="x")

    SAVE_BUTTON = ctk.CTkButton(
        buttons_frame, text="↓ Save Board as Image", state="disabled",
fg_color="transparent", border_width=1, border_color="gray",
text_color="white",
height=32, font=("Poppins", 14), command=saveResultHandler
    )
    SAVE_BUTTON.pack(padx=20, pady=5, fill="x")

```

```

        SAVE_BUTTON.bind("<Enter>", lambda e:
SAVE_BUTTON.configure(text_color="white", fg_color="#5e4db2") if
SAVE_BUTTON.cget("state") == "normal" else None)
        SAVE_BUTTON.bind("<Leave>", lambda e:
SAVE_BUTTON.configure(text_color="#7b68ee", fg_color="transparent") if
SAVE_BUTTON.cget("state") == "normal" else None)

    RESET_BUTTON = ctk.CTkButton(buttons_frame, text="↺ Reset",
state="disabled",
                                fg_color="transparent", hover_color="#7b68ef",
border_width=1, border_color = "grey",
                                height=32, font=("Poppins", 14),
text_color="gray", command=lambda: resetHandler(OPT_SWITCH))
    RESET_BUTTON.pack(padx=20, pady=5, fill="x")

    buttons_frame.pack(side="bottom", fill="x", pady=(0,20))

def outputSection(content_frame):
    output_frame = ctk.CTkFrame(content_frame, fg_color="transparent",
width=490, height=480)
    output_frame.pack(side="left", padx=10, pady=10)
    output_frame.pack_propagate(False)
    statsSection(output_frame)
    visualSection(output_frame)

def visualSection(output_frame):
    global DISPLAY_AREA, SEARCHING_STATUS
    visual_frame = ctk.CTkFrame(output_frame, corner_radius=12)
    visual_frame.pack(side="top", fill="both", expand=True, pady=(0,
10))

    ctk.CTkLabel(visual_frame, text="Visualization", font=("Poppins",
15, "bold")).pack(pady=10, padx=25, anchor="w")

    DISPLAY_AREA = ctk.CTkFrame(visual_frame, fg_color="#1E1E1E",
corner_radius=10, width=300, height=300)
    DISPLAY_AREA.pack(padx=25, pady=(5, 5))
    DISPLAY_AREA.pack_propagate(False)

    placeholder = ctk.CTkLabel(DISPLAY_AREA, text="No board loaded
yet", text_color="gray", font=("Poppins", 12))
    placeholder.place(relx=0.5, rely=0.5, anchor="center")

    SEARCHING_STATUS = ctk.CTkLabel(visual_frame, text="Upload a file
to start searching", font=("Poppins", 12), text_color="gray",
anchor="w")
    SEARCHING_STATUS.pack()

```

```

def statsSection(output_frame):
    global SEARCHING_TIME, CASES_CHECKED

    stats_frame = ctk.CTkFrame(output_frame, corner_radius=12,
height=90)
    stats_frame.pack(side="bottom", fill="x")
    stats_frame.pack_propagate(False)

    stats_frame.columnconfigure((0, 1), weight=1)

    t_box = ctk.CTkFrame(stats_frame, fg_color="transparent")
    t_box.grid(row=0, column=0, pady=10)
    ctk.CTkLabel(t_box, text="Execution Time", font=("Poppins", 10,
"bold"), text_color="#7b68ee").pack()
    SEARCHING_TIME = ctk.CTkLabel(t_box, text="0", font=("Poppins",
20, "bold"))
    SEARCHING_TIME.pack()

    n_box = ctk.CTkFrame(stats_frame, fg_color="transparent")
    n_box.grid(row=0, column=1, pady=10)
    ctk.CTkLabel(n_box, text="Total Cases Checked", font=("Poppins",
10, "bold"), text_color="#7b68ee").pack()
    CASES_CHECKED = ctk.CTkLabel(n_box, text="0", font=("Poppins", 20,
"bold"))
    CASES_CHECKED.pack()

def mainApp():
    main_frame = ctk.CTkFrame(app, fg_color="transparent")

    main_frame.pack(expand=True, fill="both")

    center_container = ctk.CTkFrame(main_frame,
fg_color="transparent")
    center_container.place(relx=0.5, rely=0.5, anchor="center")

    header(center_container)
    contentSection(center_container)

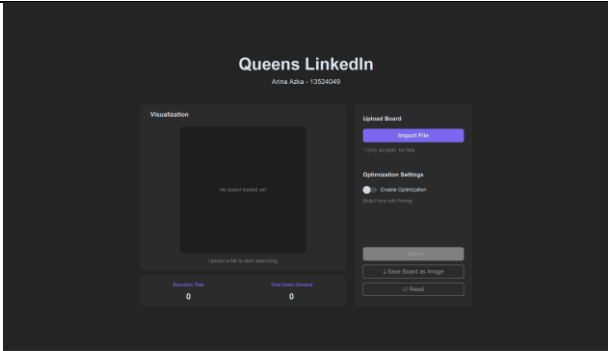
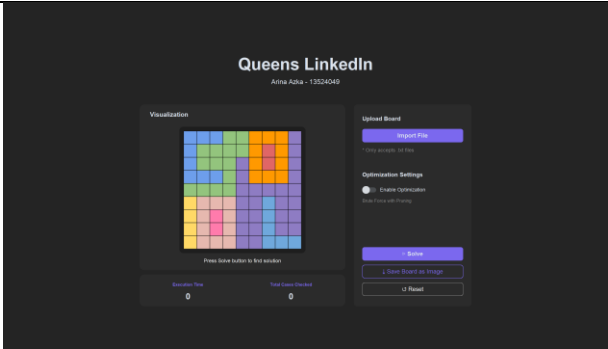
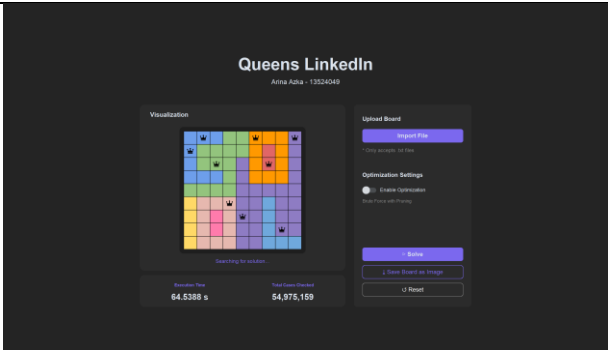
# PROGRAM UTAMA
app = ctk.CTk()
ctk.set_appearance_mode("dark")
app.geometry("880x800")
app.title("Queens LinkedIn")

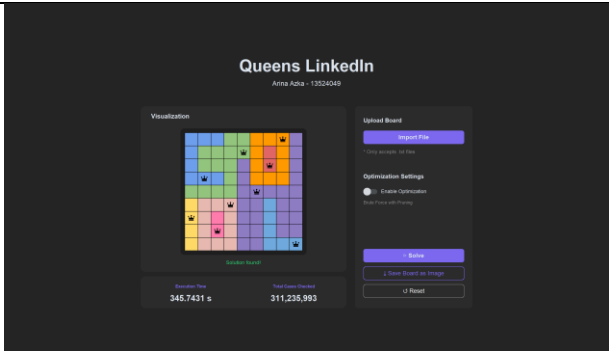
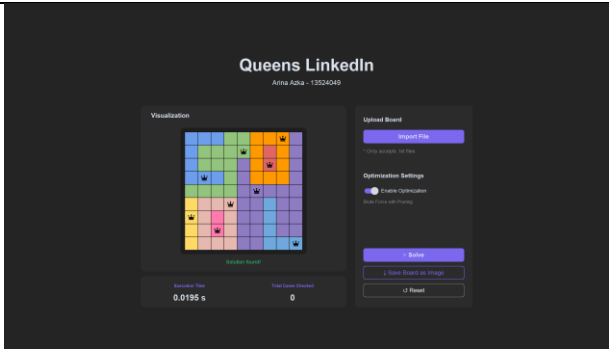
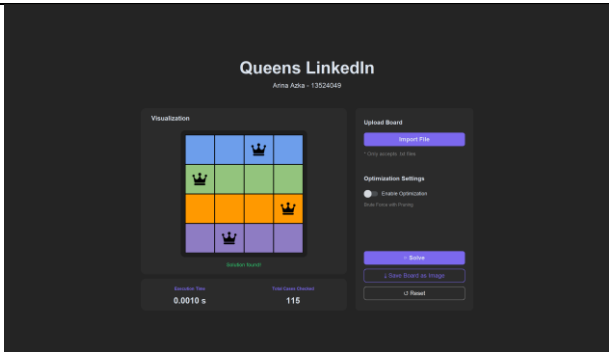
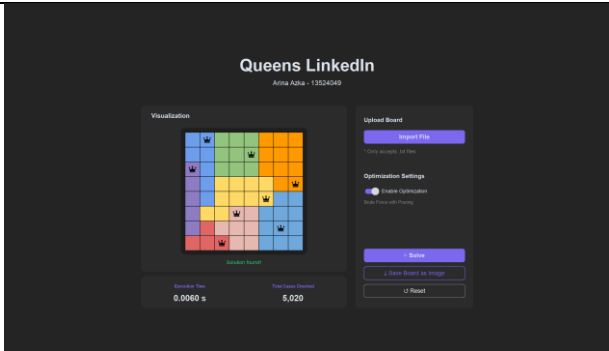
mainApp()

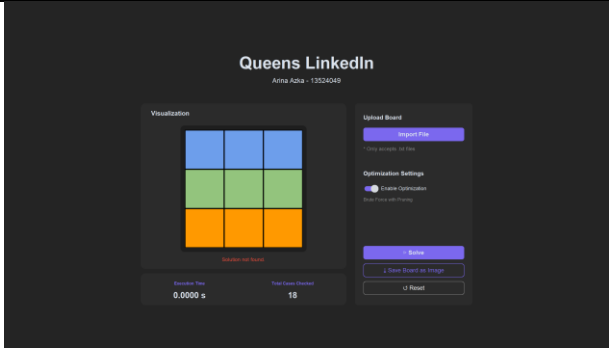
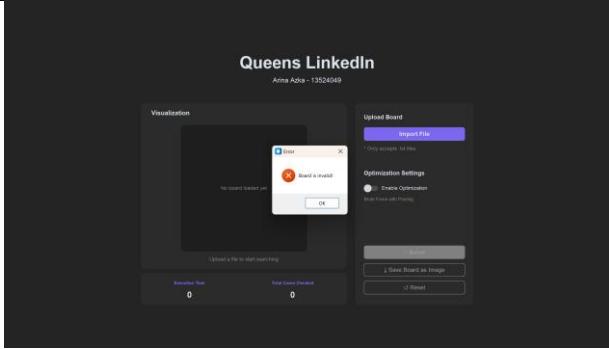
app.mainloop()

```

Tangkapan Layar

Input	Output	Keterangan
-		Tampilan awal ketika aplikasi dibuka, belum ada input yang dimasukkan
AAABBCCCD ABBBBCECD ABBBDCECD AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika mengunggah file input berisi board yang valid (.txt).
AAABBCCCD ABBBBCECD ABBBDCECD AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika aplikasi sedang melakukan pencarian tanpa optimasi.

AAABBCCCD ABBBBCECD ABBBDCECD AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika aplikais berhasil menemukan solusi untuk tc1 dengan perncarian yang tidak dioptimasi.
AAABBCCCD ABBBBCECD ABBBDCECD AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika aplikasi berhasil menemukan solusi untuk tc1 dengan perncarian yang dioptimasi.
AAAA BBBB CCCC DDDD		Tampilan ketika aplikasi berhasil menemukan solusi untuk tc2 dengan perncarian yang tidak dioptimasi.
AABBBCCC AABBBCCC DABBBCCC DAFFFFCC DAFFFFHH DFFGGHHH DEGGGHHH EEEGGHHH		Tampilan ketika aplikasi berhasil menemukan solusi untuk tc3 dengan perncarian yang dioptimasi.

AAA BBB CCC		Tampilan ketika aplikasi gagal menemukan solusi untuk tc4 dengan pencarian yang dioptimasi.
AAABBCCCD ABBBBCECDA ABBBDCEDD AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika mengunggah file input (tc5) berisi board yang tidak valid.

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Arina Azka

Appendiks

https://github.com/arinazk/Tucil1_13524049

Referensi

[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-(2026)-Bag1.pdf)

<https://www.joelotz.com/blog/2020/n-Queens-puzzle-part-2-brute-force-algorithm.html>
python-pillow.github.io

<https://www.geeksforgeeks.org/python/convert-a-numpy-array-to-an-image/>

<https://www.pythoninformer.com/python-libraries/numpy/numpy-and-images/>

<https://medium.com/@whyamit101/step-by-step-guide-to-convert-numpy-array-to-pil-image-f7f8492cd785>

<https://customtkinter.tomschimansky.com/>

https://www.youtube.com/watch?v=Miydkti_QVE

<https://youtu.be/aRJXC8hJvrc?si=y6JAcKiWBecshqsb>

Lampiran

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainannya	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	